

ASSIGNMENT 1

Web Development Fundamentals

Name: _____ Batch: _____ Date: _____

Q1. Difference between Frontend, Backend, and Full-Stack Development

Frontend development refers to everything a user sees and interacts with directly in the browser — the layout, design, buttons, forms, and animations. It is built primarily using HTML (structure), CSS (styling), and JavaScript (interactivity), often with frameworks such as React, Angular, or Vue. For example, the visual design and navigation menu of Amazon's website, or the swipe interface of Instagram, are frontend work.

Backend development refers to the server-side logic that powers an application — handling requests, processing business logic, communicating with databases, and returning data to the frontend. Common backend technologies include Node.js, Python (Django/Flask), Java (Spring Boot), and PHP. For example, when you place an order on Amazon, the backend verifies stock, processes payment, and updates the database.

Full-stack development combines both frontend and backend skills, allowing a developer to build a complete, working application end-to-end. A full-stack developer at a startup, for instance, might design the UI, write the API endpoints, and manage the database, all by themselves.

Q2. Diagram: How the Client-Server Model Works

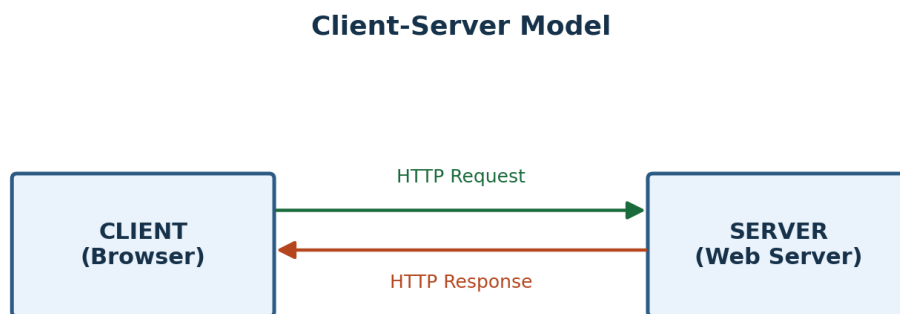


Figure 1: Client-Server communication using HTTP request/response.

In this model, the **client** (a web browser or mobile app) initiates communication by sending an HTTP request to the **server**. The server processes the request and sends back an HTTP response, usually containing HTML, CSS, JS, or data in JSON format, which the client then renders or uses.

Q3. How a Browser Requests and Displays a Web Page

- 1 The user types a URL (e.g., `www.example.com`) into the browser's address bar and presses Enter.
- 2 The browser performs a DNS lookup to translate the domain name into the server's IP address.

- 3 The browser opens a TCP connection to the server (using HTTPS/TLS for secure sites) and sends an HTTP GET request.
- 4 The web server receives the request, processes it (running any backend code, querying a database if needed), and sends back an HTTP response containing the HTML document (plus links to CSS, JS, and images).
- 5 The browser's rendering engine parses the HTML to build the DOM, parses CSS to build the CSSOM, combines them into a render tree, and paints the page on screen.
- 6 The browser then requests any additional resources referenced in the HTML (images, stylesheets, scripts) and executes JavaScript, which may further update the page dynamically.

Q4. Tools Required to Set Up a Web Development Environment

Tool	Purpose
Code Editor (e.g., VS Code)	Write, edit, and organize source code with syntax highlighting, extensions, and debugging support.
Web Browser (e.g., Chrome)	Run and preview the web page; use built-in Developer Tools to inspect elements, debug JS, and monitor network requests.
Version Control (Git)	Track changes to code over time and enable collaboration through commits, branches, and pull requests.
Git Hosting Platform (GitHub)	Host remote repositories online, allowing teams to push/pull code, review pull requests, and manage issues.
Node.js & npm	Run JavaScript outside the browser and install/manage third-party packages and build tools.
Package Manager / Bundler (e.g., Webpack, Vite)	Bundle, minify, and optimize project files for production.
Terminal / Command Line	Execute commands to run servers, install dependencies, and use Git.
Browser Extensions (e.g., React/Vue DevTools)	Provide extra debugging capability specific to a framework.

Q5. What is a Web Server? Examples of Commonly Used Servers

A **web server** is software (often running on dedicated hardware) that stores, processes, and delivers web pages to clients over HTTP/HTTPS. It listens for incoming requests from browsers, retrieves or generates the requested content (static files or dynamically generated pages), and sends the response back to the client.

Commonly used web servers include:

- **Apache HTTP Server** – one of the oldest and most widely used open-source web servers.
- **Nginx** – known for high performance, often used as a reverse proxy and load balancer.
- **Microsoft IIS (Internet Information Services)** – used for hosting ASP.NET applications on Windows.
- **Node.js (Express)** – a JavaScript runtime commonly used to build lightweight custom servers.
- **LiteSpeed** – a high-performance commercial alternative to Apache/Nginx.

Q6. Roles: Frontend Developer, Backend Developer, Database Administrator

Role	Key Responsibilities
Frontend Developer	Builds the user interface using HTML, CSS, and JavaScript; ensures the site is responsive, accessible, and easy to use.
Backend Developer	Designs and implements server-side logic, APIs, authentication, and business rules; connects the frontend to the database.
Database Administrator (DBA)	Designs the database schema, manages storage and indexing, ensures data integrity and security, and optimizes database performance.

Q7. Install VS Code and Configure it for HTML, CSS, JavaScript

Steps followed to install and configure VS Code:

- 1 Downloaded and installed Visual Studio Code from the official site: code.visualstudio.com.
- 2 Opened VS Code and installed useful extensions from the Extensions Marketplace, such as **Live Server** (to preview HTML pages with auto-reload), **Prettier** (code formatter), and **ESLint** (JavaScript linting).
- 3 Created a project folder containing an index.html file, style.css, and script.js, and opened the folder in VS Code using File > Open Folder.
- 4 Verified the setup by right-clicking index.html and selecting 'Open with Live Server' to confirm the page renders correctly in the browser.

[Insert your own screenshot here showing VS Code open with your project folder, index.html, and the Live Server extension installed, as requested by the assignment.]

Q8. Static vs Dynamic Websites

A **static website** displays fixed content that is the same for every visitor and does not change unless a developer manually edits the source files. It is built with plain HTML/CSS and loads faster since there is no server-side processing. Example: a simple personal portfolio page or a company brochure site.

A **dynamic website** generates content on the fly based on user interaction, database data, or other conditions, using server-side languages (like PHP, Python, or Node.js) alongside a database. Example: Amazon's product pages, which show different content depending on the logged-in user, search query, and live stock/price data.

Q9. Five Web Browsers and How Rendering Engines Differ

Browser	Rendering Engine
Google Chrome	Blink
Mozilla Firefox	Gecko
Apple Safari	WebKit
Microsoft Edge	Blink (Chromium-based)
Opera	Blink (Chromium-based)

A **rendering engine** is the component that interprets HTML, CSS, and JavaScript to draw the page on screen. Engines differ in how strictly and quickly they parse standards, which can cause the same page to look or behave slightly differently across browsers. Blink (used by Chrome, Edge, and Opera) is fast and has broad support for the latest web features since it is developed by Google as part of the open-source Chromium project. Gecko (Firefox) is developed by Mozilla with a strong focus on standards compliance and privacy features. WebKit (Safari) is optimized for Apple devices and tends to prioritize battery efficiency, though it has sometimes lagged in adopting newer web APIs compared to Blink.

Q10. Labeled Diagram: Basic Web Architecture Flow

Basic Web Architecture Flow

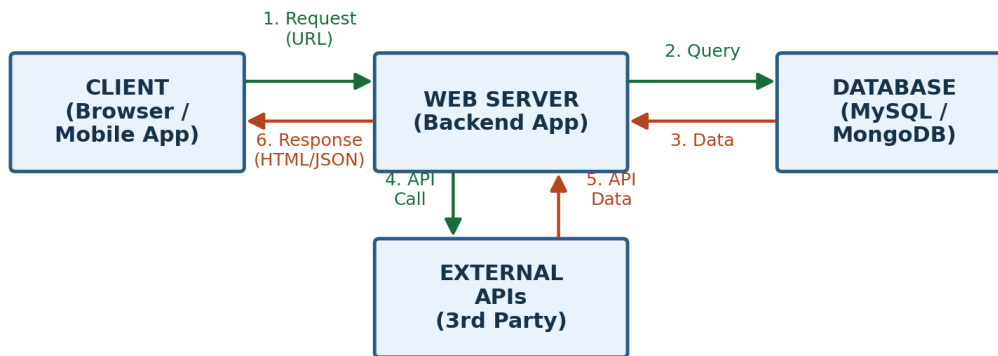


Figure 2: Flow between Client, Web Server, Database, and External APIs.

Flow explanation: (1) The client sends a request to the web server. (2) The server queries the database for required data. (3) The database returns the data to the server. (4) If needed, the server calls an external API for additional data/services. (5) The external API returns its data. (6) The server compiles everything into a response (HTML/JSON) and sends it back to the client, which renders the final page.